Assignment - SQL Joins

Table Structures

-- Table: Students

CREATE TABLE Students (

student_id INT PRIMARY KEY,

student_name VARCHAR(50),

course_id INT

);

INSERT INTO Students VALUES

(1, 'Arjun Mehta', 101),

(2, 'Priya Sharma', 102),

(3, 'Ravi Kumar', 101),

(4, 'Sneha Iyer', 103),

(5, 'Vikram Singh', NULL);

-- Table: Courses

CREATE TABLE Courses (

course_id INT PRIMARY KEY,

course_name VARCHAR(50),

```
    teacher_id INT
);

INSERT INTO Courses VALUES
(101, 'Mathematics', 1),
(102, 'Physics', 2),
(104, 'Biology', 3);

-- Table: Teachers
CREATE TABLE Teachers (
    teacher_id INT PRIMARY KEY,
    teacher_name VARCHAR(50)
);
INSERT INTO Teachers VALUES
(1, 'Anita Desai'),
(2, 'Rahul Verma'),
(3, 'Sunita Kapoor'),
(4, 'Ajay Malhotra');
```

Questions

1. **INNER JOIN** – Display all students along with their course names (only those who have a valid course).

2. **LEFT JOIN** – List all students and their course names, including those who haven't been assigned a course.

3. **RIGHT JOIN** – Display all courses and the students enrolled in them, including courses with no students.

4. **FULL JOIN** – Show all students and courses, matching where possible.

5. **JOIN with multiple tables** – Show each student's name, course name, and teacher name (only for students who have a course and a teacher assigned).

6. **LEFT JOIN with NULL filter** – Find students who have **no course assigned**.

7. **RIGHT JOIN with NULL filter** – Find courses that **have no students enrolled**.

8. **CROSS JOIN** – Generate all possible student–course combinations.

9. **SELF JOIN** – Assume students in the same course are "classmates". Display all possible student pairs who are classmates.

10. **JOIN with condition** – Display all students enrolled in a course taught by `Anita Desai`.
